

Capstone Project Assignment
DevOps & Automation Lab (ENSP461)
B. Tech CSE – Semester VII

Course: DevOps & Automation Lab

Course Code: ENSP461

Credits: 1 (0-0-2)

Assignment Type: Capstone Project

Duration: Based on Laboratory Sessions and Should be Completed during the semester

Maximum Marks: 100

This capstone project is designed directly from the official laboratory syllabus and integrates Git, Jenkins, Docker, Kubernetes, Ansible, Terraform, monitoring, security, GitOps, deployment strategies, and DevSecOps practices into one end-to-end solution.

Project Title

Design and Implementation of an End-to-End DevOps Pipeline for Cloud-Native Application Deployment

Problem Statement

Modern software organizations require applications to be delivered rapidly, securely, and reliably. Manual deployment processes are slow, error-prone, and difficult to scale. Organizations therefore adopt DevOps practices that automate software development, testing, deployment, monitoring, and infrastructure provisioning.

In this project, students will design, implement, and demonstrate a complete DevOps pipeline using industry-standard open-source tools.

Project Objective

Develop a complete automated DevOps workflow that includes

- Version Control
- Continuous Integration
- Continuous Deployment

- Containerization
- Kubernetes Deployment
- Infrastructure Automation
- Monitoring
- Logging
- Security
- GitOps
- Deployment Strategies

Suggested Application

Students may choose **one** of the following applications:

- Student Management System
- Library Management System
- Online Food Ordering
- Hospital Management System
- Weather Dashboard
- Employee Management System
- Inventory Management
- Notes Application
- Blog Website
- Task Manager

The application may be developed using

- Python Flask
- Node.js
- Java Spring Boot
- PHP
- Django

Functional Requirements

The application shall support at least

- User Login
- CRUD Operations
- Database Connectivity
- Responsive Web Interface

- REST APIs
- Error Handling

DevOps Requirements

Phase 1: Version Control (Git)

Students shall

- Create Git Repository
- Use GitHub
- Create Branches
- Merge Branches
- Resolve Merge Conflicts
- Maintain Commit History

Deliverable

GitHub Repository URL

Phase 2: Continuous Integration (Jenkins)

Implement

- Automatic Build
- Automatic Testing
- Automatic Packaging

Pipeline must execute automatically after every Git Push.

Phase 3: Docker

Students shall

Create

- Dockerfile
- Docker Image
- Docker Compose

Application must run completely inside Docker Containers.

Phase 4: Kubernetes

Deploy application using

- Deployment
- Service
- ConfigMap
- Secret

- Persistent Volume

Demonstrate

- Scaling
- Rolling Update
- Rollback

Phase 5: Ansible

Create Playbooks for

- Docker Installation
- Kubernetes Installation
- Application Deployment
- Configuration Management

Phase 6: Terraform

Provision infrastructure using

- Variables
- Outputs
- Modules

Infrastructure should be reproducible.

Phase 7: Monitoring

Configure

- Prometheus
- Grafana Dashboard

Monitor

- CPU
- Memory
- Container Health
- Application Metrics

Phase 8: Logging

Configure centralized logging using

- ELK Stack

OR

- Fluentd

Collect

- Application Logs
- Container Logs
- Kubernetes Logs

Phase 9: Security

Implement

- SonarQube Code Analysis
- Secret Management
- Secure Docker Images
- Role-Based Access Control
- Secure Credentials in Jenkins

Phase 10: Deployment Strategy

Implement one deployment strategy

- Blue-Green Deployment

OR

- Canary Deployment

Phase 11: GitOps

Deploy application using GitOps principles.

Students may use

- ArgoCD

OR

- FluxCD

Minimum Deliverables

Each team shall submit

1. GitHub Repository

Contains

- Source Code
- Dockerfile
- Jenkinsfile
- Terraform Files
- Kubernetes YAML
- Ansible Playbooks
- Documentation

2. Technical Report

Maximum 20 Pages

Must include

- Problem Statement
- Objectives
- Architecture
- Tools Used
- Screenshots
- Results
- Challenges
- Future Scope

3. Architecture Diagram

Illustrating

Developer



GitHub



Jenkins



Docker



Kubernetes



Monitoring



Logging



End User

4. Demonstration Video (If Possible)

10–15 Minutes

5. Presentation

10 Minutes

Team Size

- Individual

OR

- Maximum 2 Students

Project Timeline

Week Activity

Week 1 Git + GitHub + Application Setup

Week 2 Jenkins + Docker

Week 3 Kubernetes + Ansible

Week 4 Terraform + Monitoring

Week 5 Logging + Security

Week 6 GitOps + Deployment Strategy

Week 7 Final Integration + Documentation + Demo

Evaluation Rubric

Component	Marks
Application Development	15
Git Repository Management	10
Jenkins CI/CD Pipeline	10
Docker Implementation	10
Kubernetes Deployment	15
Ansible Automation	10
Terraform Infrastructure	10
Monitoring & Logging	5
Security Implementation	5
GitOps / Deployment Strategy	5
Documentation	3

Component	Marks
Demonstration &	2
Total	100

Bonus Features (Up to 10 Extra Marks)

Students may implement any of the following:

- AI-based Log Analysis
- Slack/Email Notifications
- Auto Scaling
- Helm Charts
- Kubernetes Ingress
- CI/CD Security Scanning
- Unit Test Coverage Reports
- Automated Backup
- Multi-Container Microservices
- Cloud Deployment (AWS, Azure, GCP)

Learning Outcomes

Upon successful completion of the capstone project, students will be able to:

- Apply collaborative software development using Git.
- Design and implement automated CI/CD pipelines with Jenkins.
- Containerize applications using Docker and orchestrate them with Kubernetes.
- Automate infrastructure provisioning using Ansible and Terraform.
- Implement monitoring, logging, and security practices for cloud-native applications.
- Demonstrate GitOps workflows and advanced deployment strategies such as Blue-Green or Canary deployments.
- Deliver an end-to-end DevOps solution that reflects industry-standard software delivery practices.

This assignment comprehensively integrates all five course outcomes (CO1–CO5) and the laboratory